

Hands on for SVM

Experiment Objectives:

Master the principles and learning algorithm of SVM.

Experiment Requirements:

Implement the SVM learning algorithm for Classification and Regression tasks, respectively.

Part 1: Classification

Implement the SVM learning algorithm with different choices of kernel value (Linear SVM and Nonlinear SVM).

Answer the question: What are the differences between the different kernel values?

Optional step: implement the SVM learning algorithm with different choices of "C" value.

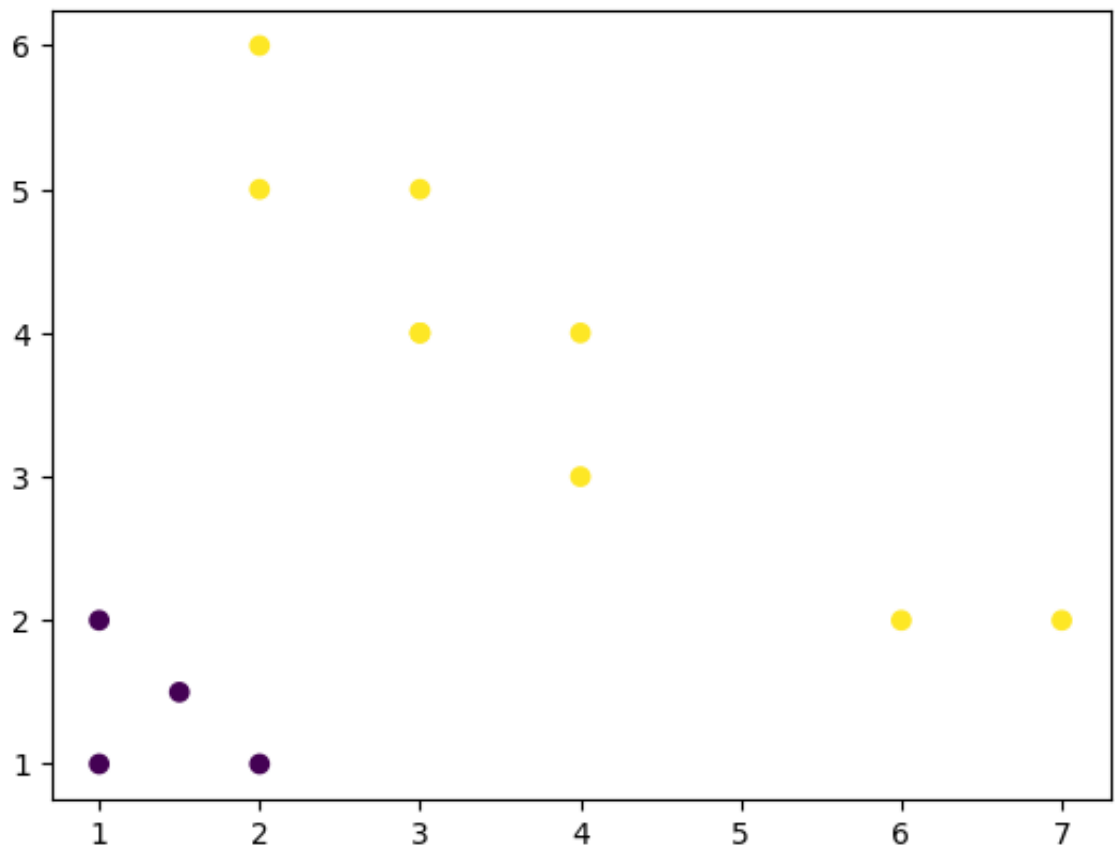
Note: Please replace "random_state" in the code with the last four digits of your student ID.

```
In [23]: import numpy as np
         from sklearn.svm import SVC
         import matplotlib.pyplot as plt
```

```
In [24]: X = np.array([[1,1],[2,1],[1,2],[1.5,1.5],[3,4],[2,5],[4,3],[7,2],[
         y = [0,0,0,0,1,1,1,1,1,1,1,1]
```

```
In [ ]:
```

```
In [25]: X_x1 = X[:,0]
X_x2 = X[:,1]
plt.scatter(X_x1, X_x2, c = y)
plt.show()
```



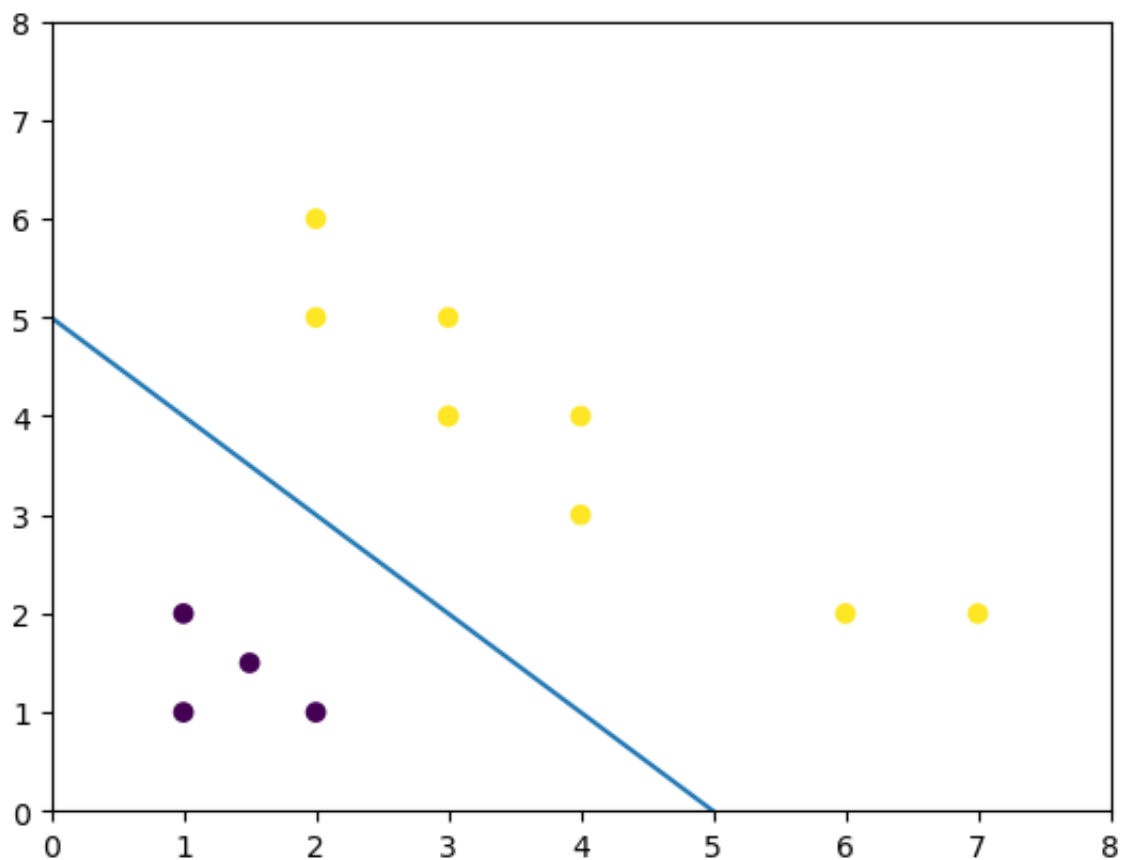
```
In [26]: svcLinear = SVC(kernel='linear').fit(X, y)
svcLinear.coef_, svcLinear.intercept_
```

```
Out[26]: (array([[0.49989823, 0.49989823]]), array([-2.49953185]))
```

$$g(x) = w_1x_1 + w_2x_2 + w_0$$

$$x_2 = -(w_0 + w_1x_1)/w_2$$

```
In [27]: x1 = np.array([0, 5])
x2 = -1 * (svcLinear.intercept_ + svcLinear.coef_[0][0] * x1) / svcLin
plt.plot(x1, x2)
plt.scatter(X_x1, X_x2, c = y)
plt.axis([0, 8, 0, 8])
plt.show()
```



Classification on iris dataset

```
In [28]: from sklearn import svm, datasets
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

```
In [29]: data = datasets.load_iris()
x = data.data[:,0:2]
y = data.target
```

```
In [30]: x_train,x_test,y_train,y_test = train_test_split(x,y,random_state=4)
```

```
In [31]: y_train
```

```
Out[31]: array([0, 0, 2, 1, 1, 0, 0, 1, 2, 2, 1, 2, 1, 2, 1, 0, 2, 1, 0, 0,
 0, 1,
           2, 0, 0, 0, 1, 0, 1, 2, 0, 1, 2, 0, 2, 2, 1, 1, 2, 1, 0, 1,
 2, 0,
           0, 1, 1, 0, 2, 0, 0, 1, 1, 2, 1, 2, 2, 1, 0, 0, 2, 2, 0, 0,
 0, 1,
           2, 0, 2, 2, 0, 1, 1, 2, 1, 2, 0, 2, 1, 2, 1, 1, 1, 0, 1, 1,
 0, 1,
           2, 2, 0, 1, 2, 2, 0, 2, 0, 1, 2, 2, 1, 2, 1, 1, 2, 2, 0, 1,
 2, 0,
           1, 2])
```

```
In [32]: train_test_split?
```

Here, SVM() uses one Vs one multiclass classification

```
In [33]: clf1 = svm.SVC(C = 1, kernel = 'linear')
clf1.fit(x_train, y_train)
```

```
Out[33]: SVC
SVC(C=1, kernel='linear')
```

```
In [34]: clf1.score(x_test, y_test)
```

```
Out[34]: 0.8421052631578947
```

```
In [35]: def makegrid(x1, x2, h = 0.02):
          x1_min, x1_max = x1.min() - 1, x1.max() + 1
          x2_min, x2_max = x2.min() - 1, x2.max() + 1
          a = np.arange(x1_min, x1_max, h)
          b = np.arange(x2_min, x2_max, h)
          xx, yy = np.meshgrid(a, b)
          return xx, yy
```

Decision Boundary using linear kernel

```
In [36]: xx, yy = makegrid(x[:, 0], x[:, 1])
predictions = clf1.predict(np.c_[xx.ravel(), yy.ravel()])
plt.scatter(xx.ravel(), yy.ravel(), c = predictions)
plt.show()
```



Nonlinear SVM

```
In [37]: clf = svm.SVC(gamma='auto') #RBF kernel
clf.fit(x_train,y_train)
```

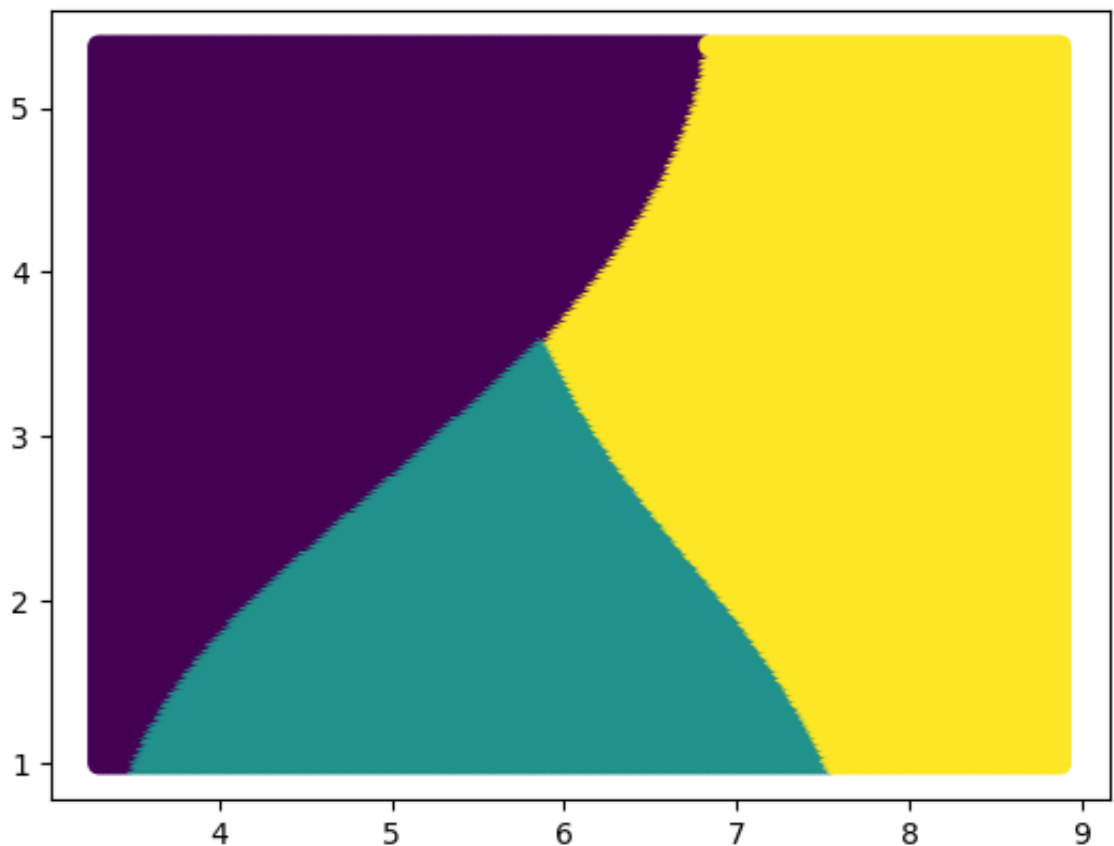
```
Out [37]: SVC
SVC(gamma='auto')
```

```
In [38]: clf.score(x_test,y_test)
```

```
Out [38]: 0.8421052631578947
```

Decision Boundary using RBF or Gaussian kernel

```
In [39]: xx, yy = makegrid(x[:, 0], x[:, 1])
predictions = clf.predict(np.c_[xx.ravel(), yy.ravel()])
plt.scatter(xx.ravel(), yy.ravel(), c = predictions)
plt.show()
```



Part 2: Regression

Implement the SVR learning algorithm with different choices of kernel value (Linear SVR and Nonlinear SVR).

Answer the question: What are the differences between the different kernel values?

Optional step: implement the SVR learning algorithm with different choices of "C" value and "gamma" value, respectively.

```
In [40]: from sklearn.svm import SVR
import matplotlib.pyplot as plt
```

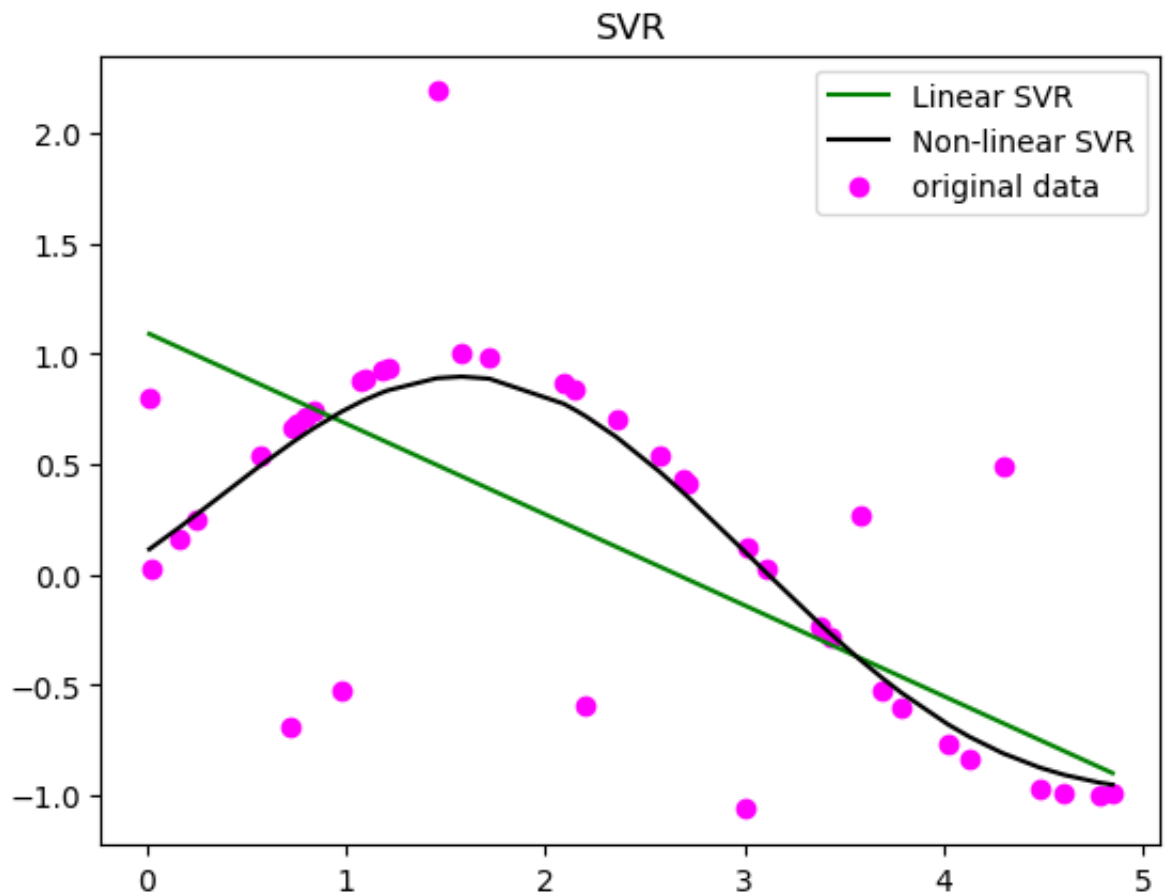
```
In [41]: # Generate sample data
X = np.sort(5 * np.random.rand(40, 1), axis=0)
y = np.sin(X).ravel()
```

```
In [42]: # Add noise to targets
y[::5] += 3 * (0.5 - np.random.rand(8))
```

```
In [43]: svr_lin = SVR(kernel='linear', C=100, gamma='auto')
svr_rbf = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=.1)
svr_lin.fit(X,y)
svr_rbf.fit(X,y)

y_lin = svr_lin.predict(X)
y_rbf = svr_rbf.predict(X)
```

```
In [44]: # Visualization
plt.plot(X, y_lin, color = 'green')
plt.plot(X, y_rbf, color = 'black')
plt.scatter(X,y,color = 'magenta')
plt.title('SVR')
plt.legend(['Linear SVR', 'Non-linear SVR', 'original data'])
plt.show()
```



```
In [ ]:
```